

Connexion avant accès

La plateforme exige une connexion avant d'afficher quoi que ce soit. Deux portes :

- **Google** : si la section `[auth]` est configurée, un bouton « Se connecter avec Google » apparaît.
- **Compte CHRUTH** : toujours disponible. Sans Google configuré, c'est la seule porte — chaque membre crée son compte (adresse + mot de passe) depuis l'écran de connexion, puis se connecte. Le mot de passe n'est jamais stocké en clair : seul son hachage vit dans l'espace chiffré du membre.

Dans les deux cas, l'application ne répond qu'à une question : **cette adresse a-t-elle le droit d'entrer ?**

La réponse vit dans `.streamlit/secrets.toml`, un fichier que seul l'administrateur peut modifier — un droit d'accès modifiable depuis l'application protégée ne protégerait rien. Sans `[acces]` (poste local), tout compte authentifié entre.

1. Créer l'identifiant Google (une seule fois, ~10 minutes)

1. Ouvrir <https://console.cloud.google.com/> et créer un projet (nom libre).
2. Menu **APIs et services** → **Ecran de consentement OAuth**. - Type **Externe**, renseigner le nom de l'application et un email de contact. - Dans **Utilisateurs test**, ajouter les adresses qui se connecteront tant que l'application n'est pas publiée.
3. Menu **Identifiants** → **Créer des identifiants** → **ID client OAuth**. - Type : **Application Web**. - **URI de redirection autorisés** : `http://localhost:8501/oauth2callback` (pour l'application en ligne, ajouter aussi son adresse suivie du même suffixe — voir l'étape 4).
4. Copier l'**ID client** et le **Code secret du client**.

2. Remplir le fichier de secrets

```
cp .streamlit/secrets.toml.template streamlit/secrets.toml
python -c "import secrets; print(secrets.token_hex(32))" # pour cookie_secret
```

Puis renseigner dans `.streamlit/secrets.toml` : `cookie_secret`, `client_id`, `client_secret`, et les adresses dans `[acces]`.

```
[acces]
emails = ["collaboratrice@exemple.fr"] # liste VIDE = tout compte Google entre
admins = ["ton.adresse@exemple.fr"]
```

3. Verifier

```
streamlit run CHRUTH_APP.py
```

Attendu : un ecran « Se connecter », puis la plateforme. La barre laterale affiche l'adresse connectee et un bouton **Se deconnecter**.

4. Activer la connexion sur l'application en ligne

L'application deployee sur Streamlit Community Cloud ne lit pas `.streamlit/secrets.toml` : ce fichier n'est pas envoye, et c'est voulu. Les memes valeurs se saisissent dans le tableau de bord, ou seul le proprietaire du compte entre.

L'ecran de connexion, lui, s'affiche de toute facon : il ne depend d'aucun reglage. Ce que les secrets decident, c'est **qui franchit cet ecran**. Sans section `[acces]`, la liste des autorises est vide, et une liste vide laisse entrer tout compte authentifie — or n'importe quel visiteur peut se creer un compte CHRUTH depuis l'ecran lui-meme. En ligne, l'ecran seul ne protege donc rien : c'est `emails` qui ferme la porte.

1. Google Cloud → **Identifiants** → l'ID client de l'etape 1 → **URI de redirection autorises** → ajouter l'adresse en ligne suivie de `/oauth2callback`, par exemple `https://chruth-plateforme.streamlit.app/oauth2callback`. Garder aussi l'adresse locale : les deux cohabitent dans la meme liste.
2. Ouvrir <https://share.streamlit.io>, puis l'application → menu : → **Settings** → **Secrets**.
3. Coller le contenu ci-dessous, rempli. `redirect_uri` prend ici l'adresse en ligne, pas `localhost` — c'est la difference avec le poste local.

```
[acces]
emails = ["collaboratrice@exemple.fr"]
admins = ["ton.adresse@exemple.fr"]

[auth]
redirect_uri = "https://chruth-plateforme.streamlit.app/oauth2callback"
cookie_secret = "" # python -c "import secrets; print(secrets.token_hex(32))"

[auth.google]
client_id = ""
client_secret = ""
server_metadata_url = "https://accounts.google.com/.well-known/openid-configuration"
```

4. **Save**. L'application redemarre seule. Recharger la page doit afficher l'ecran « Se connecter » a la place de l'accueil.

Tant que l'écran de consentement Google reste en mode **Test**, seules les adresses inscrites en **Utilisateurs test** peuvent se connecter, même si elles figurent dans `emails`. Deux listes, à deux endroits : une adresse absente de l'une ou de l'autre n'entre pas.

Gerer les acces au quotidien

Objectif	Geste
Ouvrir l'accès à quelqu'un	Ajouter son adresse dans <code>emails</code> , relancer l'app
Retirer un accès	Retirer son adresse de <code>emails</code> , relancer l'app
Ouvrir à tout compte	Laisser <code>emails = []</code> — à réserver au poste local
Retirer le bouton Google	Commenter toute la section <code>[auth]</code> (la connexion reste requise via les comptes locaux)

En ligne, ces gestes se font dans **Settings** → **Secrets** ; l'application redemarre seule après l'enregistrement, sans relance manuelle.

Une adresse refusée voit un message qui nomme les administrateurs à contacter : un refus sans recours transforme un réglage à corriger en impasse.

Points de vigilance

- `.streamlit/secrets.toml` est ignoré par git. Ne jamais le commiter, ne jamais le joindre à une livraison — il ouvre l'accès à l'application.
- Une liste `emails` vide est un raccourci commode sur un poste local. Sur une adresse publique, c'est une porte ouverte : la création de compte est offerte à qui arrive sur l'écran, et un compte crée entre. La base de données comme le CRM sont derrière. En ligne, la liste se remplit — c'est la seule barrière.
- Le `client_secret` se colle dans le tableau de bord Streamlit et nulle part ailleurs : ni dans le dépôt, ni dans un message, ni dans un fichier livre.
- Changer `cookie_secret` déconnecte tout le monde. C'est le geste à faire si on soupçonne une fuite.
- Une adresse en ligne qui change (renommage de l'application) invalide le `redirect_uri` : la connexion échoue tant que Google et les secrets ne portent pas la nouvelle adresse.